

Guidelines / requirements to be considered for the proposed solution – Cybersecurity and Artificial intelligence aspects

Done by:

R. Balasoupramanien

Burak Uchar



Table of Contents

A. Guideline for Secure Coding Implementation	3
B. Guidelines for Proper Handling of Data	5
C. Guidelines for Threat Awareness and Mitigation	7
D. Guidelines for Compliance to the Regulations (AI)	9
E. Guidelines for Compliance to NIQ Generative AI	9
F. Guidelines for Mitigation of Hallucination and Misinformation risks	10
G. Guidelines for Controls for Jailbreak and prompt injection attacks	10
H. Guideline for Controls for Information Leakage	11
I. Guideline for Security of the Data Sources	11
J. Guideline for Secure Plugin Design.....	12
K. Guideline for Output Validation	12

A. Guideline for Secure Coding Implementation

1. Development Environment Requirements

- Use **version-controlled source code repositories** (e.g., GitHub, GitLab)
- Enforce **multi-factor authentication (MFA)** for repo access
- Use **secured IDEs** and disallowed storing secrets in local files
- Implement **code linting and formatters** with secure defaults

2. Coding Standards & Practices

- Follow **OWASP Secure Coding Guidelines** (language-specific) – Good to have
- Apply **input validation** (e.g., whitelist input types)
- Use **output encoding** to prevent XSS
- Avoid **hardcoded credentials**, tokens, and secrets
- Ensure **secure use of cryptography** (e.g., TLS, SHA-256, AES-256)
- Prevent **SQL injection** using parameterized queries or ORM
- Protect **API keys and tokens** using environment variables or secret managers

3. Dependency and Package Management

- Use **trusted package sources** (e.g., npm, pip, Maven)
- Apply **software composition analysis (SCA)** tools (e.g., Snyk, Dependabot)
- Regularly **scan for vulnerable packages** (use **cycode** for scan for potential vulnerabilities such as SQL injection, XSS and other common web application risks)
- Pin versions to avoid untrusted updates (requirements.txt, package-lock.json)

4. Secure Authentication & Authorization

- Use **standard protocols** (e.g., OAuth 2.0, OpenID Connect)
- Implement **role-based access control (RBAC)** or **attribute-based access control (ABAC)**
- Store passwords using **strong hashing** (bcrypt, Argon2) – Good to have
- Implement **session management** with timeouts and regeneration - Good to have

5. Secure Code Review & Testing

- Conduct **peer code reviews** focusing on security
 - Integrate **Static Application Security Testing (SAST)** into CI/CD (if applicable)
 - Use **Dynamic Application Security Testing (DAST)** in staging/test environments (if applicable)
 - Perform **unit tests** for security-critical logic
 - Enforce **linting rules** for security
-

6. Secure CI/CD Integration – If applicable

- Scan code on pull requests for secrets (e.g., with GitHub Actions + TruffleHog)
 - Enforce **build-time security checks** (SAST/SCA) – Good to have
 - Use **signed builds** and **artifact integrity checks**
 - Isolate dev, test, and prod environments
 - Apply **least privilege** IAM policies to pipelines
-

7. Secrets & Configuration Management

- Store secrets in **secret management tools** (e.g., AWS Secrets Manager, Azure Key Vault)
 - Ensure not to include any secret keys or values into the code you develop
 - Separate **configuration from code**
 - Avoid secrets in logs and exception messages
-

8. Training & Governance

- Developers complete **secure coding training** (e.g., OWASP Top 10) – this training is for those do not have knowledge of secure coding
 - Establish and enforce a **secure coding policy**
 - Create a **security checklist** or secure SDLC guide for developers
 - Maintain **security champions** within dev teams
-

B. Guidelines for Proper Handling of Data

1. Encryption Requirements

Data at Rest

- Encrypt all sensitive data using **AES-256 or equivalent – good to have**
- Use **managed encryption services** (e.g., AWS KMS, Azure Key Vault, GCP CMEK)
- Never store unencrypted PII, PHI, financial data, or credentials
- Secure database volumes (e.g., RDS, EBS, SQL Server encryption)

Data in Transit

- Enforce **TLS 1.2+** for all data in transit (internal and external)
- Use **HTTPS** for all APIs and UIs
- Pin or validate TLS certificates to prevent MITM attacks
- Use **VPN or private links** for backend services

Key Management

- Never hardcode encryption keys in code
- Store and rotate keys using **secure key management systems**
- Apply **least privilege** to key access policies

2. Authentication Requirements

- Use **strong authentication mechanisms** (e.g., OAuth 2.0, SAML, OpenID Connect)
- Store passwords using **bcrypt, Argon2**, or other strong hash algorithms
- Implement **multi-factor authentication (MFA)** for sensitive data access
- Use **access tokens** (JWT with short expiration + refresh tokens)
- Ensure **session security** with expiry and regeneration on privilege change

3. Authorization and Access Control

- Enforce **role-based access control (RBAC)** or **attribute-based access control (ABAC)**
- Implement **principle of least privilege** for data access

-
- Scope API tokens and database users to **minimal read/write privileges**
 - Use **fine-grained permissions** on specific data fields (e.g., mask PII for non-privileged users)
-

4. Input Validation Requirements

- Apply **input validation at all data entry points**, including APIs, forms, and files
 - Use **whitelisting (positive validation)** wherever possible
 - Sanitize all inputs to prevent:
 - **SQL Injection**
 - **Cross-Site Scripting (XSS)**
 - **Command Injection**
 - **Path Traversal**
 - Use built-in or trusted libraries for validation (e.g., validator.js, Django forms, Joi)
-

5. Logging & Monitoring (Related to Data Handling)

- Log all access to sensitive data (read/write)
 - Mask sensitive fields in logs (e.g., passwords, SSNs)
 - Feed logs into **SIEM systems** for audit and incident response
 - Monitor for **unauthorized or unusual access patterns**
-

6. Secure Data Transfer Between Services

- Sign and verify payloads using **HMAC or asymmetric signatures**
 - Use **message queues with encryption** (e.g., SQS with SSE, Kafka with TLS)
 - Avoid passing sensitive data via URL query strings
-

7. Recommended Tools and Services – Good to have

- **Key Management:** AWS KMS, Azure Key Vault, Hashi Corp Vault
 - **Input Validation Libraries:** Joi (Node.js), Cerberus (Python), Hibernate Validator (Java) / NIQ approved libraries
 - **Authentication Frameworks:** OAuth 2.0, Auth0, Okta
-

-
- **Testing:** Burp Suite, ZAP, Postman with security tests and NIQ approved testing tools

8. Policy & Training – Good to have

- Developers must complete **secure data handling training**
- Ensure compliance with frameworks (e.g., **GDPR, HIPAA, PCI DSS** depending on data type)
- Create and enforce a **data classification policy** (what needs encryption, who can access what)

C. Guidelines for Threat Awareness and Mitigation

1. Developer Training & Awareness – Good to have

- **Mandatory secure coding training** (e.g., OWASP Top 10, SANS Top 25) – this is for those don't have knowledge of secure coding
- Regular **security awareness workshops** or threat modelling sessions
- Access to secure coding guidelines and cheat sheets
- Awareness of modern attack vectors:
 - Injection (SQL, NoSQL, LDAP)
 - Cross-site scripting (XSS)
 - Cross-site request forgery (CSRF)
 - Broken access control
 - Insecure deserialization
 - API-specific threats (rate limiting, replay attacks)

2. Threat Modelling Process

- Perform **threat modelling** early in the SDLC (e.g., STRIDE, DREAD frameworks)
- Identify **attack surfaces** and document trust boundaries

-
- Use tools like **Microsoft Threat Modelling Tool**, OWASP (Open Worldwide Application Security Project) Threat Dragon
 - Address threats with **secure design patterns** and mitigations
-

3. Secure Architecture and Design

- Use **secure architecture blueprints** for web apps, APIs, cloud deployments
 - Design systems with **least privilege**, **zero trust**, and **defense in depth**
 - Enforce **segregation of duties** (admin vs. user roles)
 - Validate assumptions about **external systems and data inputs**
-

4. Tooling for Threat Detection and Mitigation – Good to have

- Integrate **Static Application Security Testing (SAST)** into CI/CD (if applicable)
 - Use **Dynamic Application Security Testing (DAST)** for runtime threat detection (if applicable)
 - Use **Software Composition Analysis (SCA)** for vulnerable dependency detection
 - Run **dependency updates and patching alerts**
-

5. Documentation and Checklists

- Maintain a **secure development checklist** per project
 - Document common threat scenarios and resolutions per architecture pattern
 - Include **security risk acceptance and mitigations** in design reviews
-

6. Incident Readiness (for Developers)

- Developers know how to **report a security issue or bug**
 - Awareness of **bug bounty programs** or **security incident response plans**
 - Application logs security-relevant events (auth failures, access violations)
-

7. Mitigation Strategy Requirements – Good to have

For each threat, implement the following types of mitigation:

Threat Type	Mitigation Strategy
Injection (SQL, XSS, etc.)	Input validation, ORM (object-Relational Mapping), output encoding
Authentication weaknesses	MFA, rate limiting, secure password storage
Broken access control	RBAC/ABAC, enforce resource-level permissions
Data exposure	Encryption, secure API design
Denial of service (DoS)	Rate limiting, timeouts, retries
Insecure APIs	API gateways, JWT (JSON Web Token) validation, throttling

8. Security Champions

- Assign **security champions** within dev teams to evangelize security practices
- Involve them in reviewing pull requests for threat mitigation adherence

D. Guidelines for Compliance to the Regulations (AI)

- All our AI apps should establish transparency to external end users that they are interacting with an AI. Example, if you are using AI to drive a Chatbot, you need to make sure it is clear to the consumer interacting with the chatbot that AI is being used.
- If personal data, including biometric data will be used, i.e. for speech to text translation, the solution must be compliant with the privacy regulations.

E. Guidelines for Compliance to NIQ Generative AI

- For opensource models, license of the model must be approved by Legal.
- Only secure model formats (i.e. Safe tensors) are allowed to be downloaded from the Internet.
- Only approved 3rd party model hosting providers (Azure AI Services) must be consumed.

For more information:

nielsenenterprise.sharepoint.com/sites/GenerativeAI/Shared Documents/Forms/AllItems.aspx?id=%2Fsites%2FGenerativeAI%2FShared Documents%2FGenAI Internal Guidelines%2FGenAI AI Research and Development Use Case Guidelines%2Epdf&parent=%2Fsites%2FGenerativeAI%2FShared Documents%2FGenAI Internal Guidelines

F. Guidelines for Mitigation of Hallucination and Misinformation risks

LLM responses are not deterministic by nature and generated based on the outdated training data, therefore outputs of LLMs might contain incorrect information to mitigate the risk of misinformation.

- LLMs outputs must be restricted to the specific use case.
- Outputs must be verified either automatically by another LLM or by human in the loop processes for the high-risk applications.
- For the models consumed from Azure AI Services content filters must be used to filter out unwanted outputs.

G. Guidelines for Controls for Jailbreak and prompt injection attacks

Prompt injection attacks try to manipulate large language model (LLM) responses via specifically crafted inputs. Direct injections aim to overwrite or escape from system prompts, while indirect injection attacks aim to manipulate the LLMs from external sources like files e-mails or web pages.

- Model behaviour must be strictly restricted by instructions, to only allow the specific role of the model, tasks, topics and capabilities.
- Models must be instructed to ignore any attempt to modify the core instructions.

These restrictions must be placed at the end of the prompt, after the external inputs.

- Character sets of prompts from end-users should be restricted to the expected set for the application. i.e. Only alphanumeric characters and punctuation must be allowed for prompts from end-users.
- Context window must be limited to maximum characters expected by the application.
- For the models consumed from Azure AI services, content filters must be implemented against Jailbreak attacks.
- Model output must be sanitized based on the expected output.
- Prompts and responses must be logged and monitored against prompt injection attacks.

H. Guideline for Controls for Information Leakage

GenAI applications must be designed with the controls to detect and prevent Information Leakage.

- System prompts should not contain any sensitive information.
- Backends of RAG applications, either relational or vector databases, should authorize the query with the end user permissions.

I. Guideline for Security of the Data Sources

- Access to the Data Sources must be restricted with the minimum required permissions.
- Human access to the data sources with Write and Update permissions must be protected with MFA or accessible via VPN only.
- Programmatic access to the data sources with Write and Update must be protected with keys and ip restriction. Keys must be protected according to the IAM standards.
- Write / Update events to the data sources must be logged, and logs must be retained inline with the regulations.

J. Guideline for Secure Plugin Design

Plugins or extensions are functions which can be consumed by the GenAI application to perform tasks.

- Plugins must work with the access rights of the end user.
- High risk actions of LLMs, i.e. write/delete/update/execute code on backend systems, must be approved by human in the loop processes.
- Plugins must not be allowed to access the whole internet.
- Plugin code must be regularly updated in line with the patch management standards.

K. Guideline for Output Validation

LLM responses are not deterministic by nature, therefore the outputs of LLMs must not be trusted.

- Outputs must be sanitized before passing to the user browser to prevent XSS and CSRF type attacks.
- Model output should not be used for code generation. Instead, model outputs should be handled as parameters for the preapproved code.

----- End of Document -----